

This report documents an educational ransomware simulation payload developed in Kali Linux. The one-click activation program creates a text file, applies reversible XOR encryption, password-protects it with "Milestone1_Password_2024", displays ransomware and malware popups, opens 9gag.com, and logs out after 30 seconds. A desktop launcher enables one-click execution. The encryption was verified as reversible through decryption demonstration.

Ransomware Simulation for kali linux

EHA361_Milestone 4

Tshiamo Mosweu

Contents

Introduction.....	2
Virtual Machine Setup	2
Initial Challenges and Resolution.....	2
Command Entry Issues.....	2
File Extensions Issue	3
Resolution.....	4
Code Implementation	5
Core Libraries.....	5
Popup Functions	5
Encryption Implementation	6
Password Protection.....	7
Indentation Fix.....	7
One-Click Activation Setup	8
Launcher Creation Function	8
Launcher Verification.....	8
Desktop Icons.....	9
Payload Execution Results.....	9
Decryption Demonstration.....	10
Complete Payload Execution	11
Ransom Note Creation	12
Encrypted File Confirmation	12
Decryption Proof.....	13
Decryption Command Output.....	13
Decryption Algorithm.....	13
Safety Measures	13
Default Safety Mode	13
Clear Warnings.....	14
Challenges and solutions	14
Special Characters in Commands.....	14
File Extension Issue	14
Indentation Error	14
Logout Command Compatibility	14
References	15

Introduction

This report documents the development of an educational ransomware simulation created for the Milestone 4 project. The objective was to create a payload program that simulates ransomware behaviour in a controlled environment without using actual malicious code. According to Kaspersky Lab (2023), understanding ransomware mechanism through controlled simulations is essential for cybersecurity education. The payload demonstrates key cybersecurity concepts including file encryption, password protection, popup notifications, and system logout functionality.

Virtual Machine Setup

A dedicated Kali Linux virtual machine was created for this project to ensure isolation and safety, following best practice outlined in the VirtualBox documentation (Oracle Corporation, 2025).

VM Specifications:

- OS: Kali Linux (Latest)
- RAM: 4GB
- Storage: 40GB
- Network: NAT

Initial Challenges and Resolution

During the initial setup, several challenges were encountered and resolved. These challenges are common when working with command-line interfaces (Python Software Foundation, 2025).

Command Entry Issues

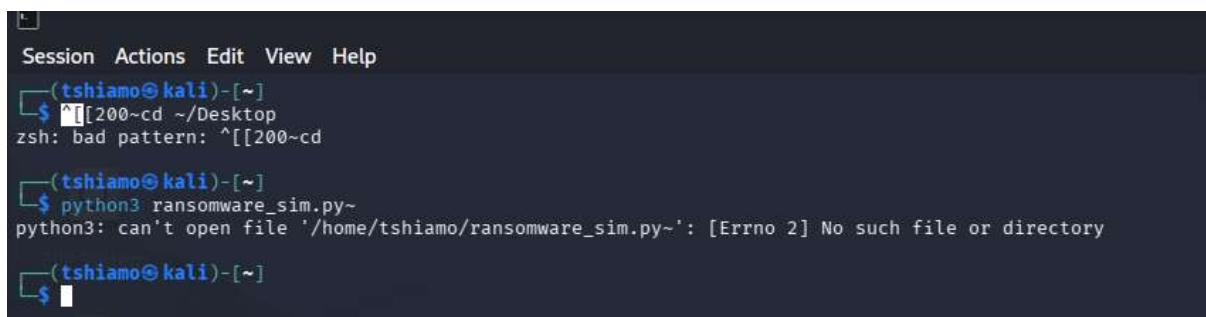
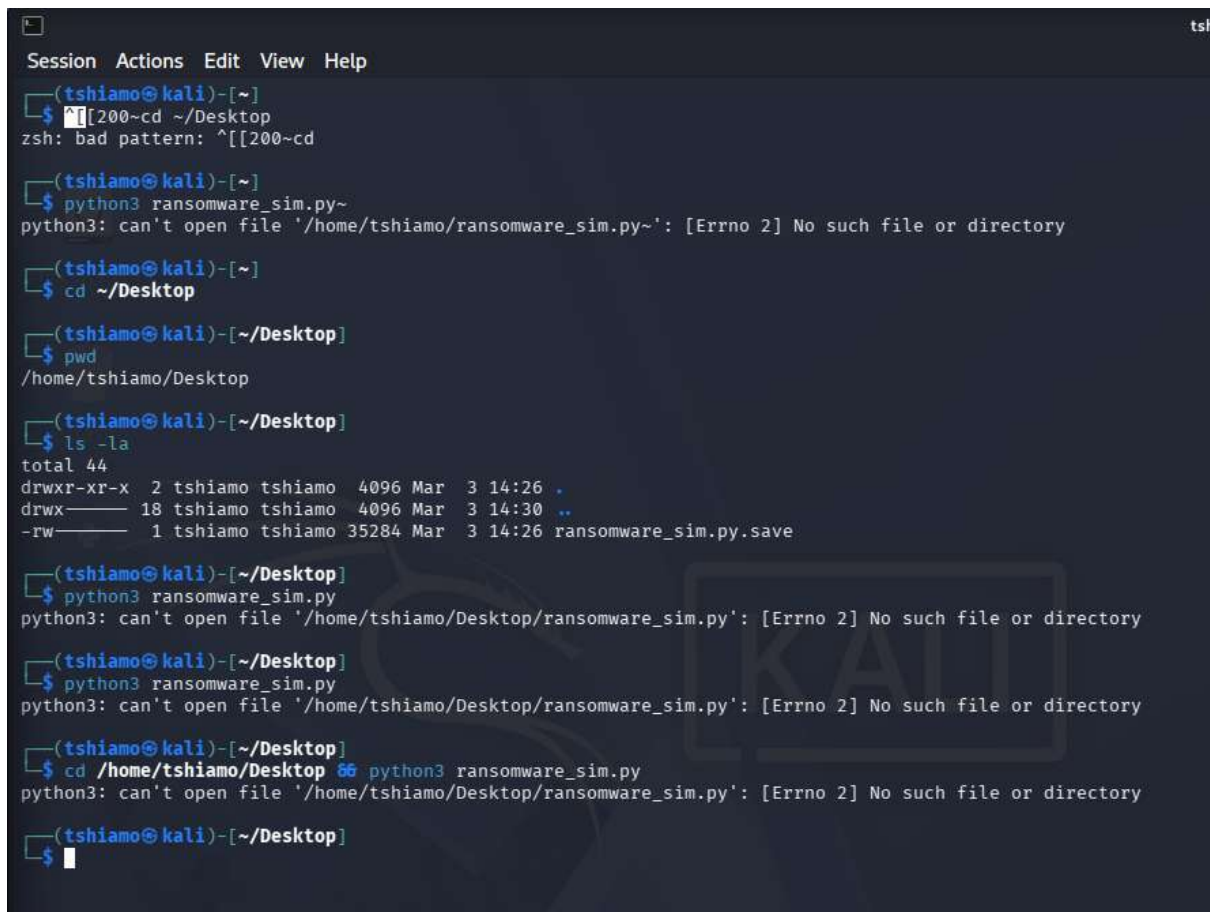
A terminal window with a dark background and light blue text. The window title bar shows 'Session Actions Edit View Help'. The prompt is '(tshiamo@kali)-[~]'. The first command entered is '\$ cd ~/Desktop', which results in an error: 'zsh: bad pattern: ^[[200~cd'. The second command entered is '\$ python3 ransomware_sim.py~', which results in an error: 'python3: can't open file '/home/tshiamo/ransomware_sim.py~': [Errno 2] No such file or directory'. The prompt returns to '(tshiamo@kali)-[~]'.

Figure 1: Terminal errors caused by copy-pasting special characters.

When first attempting to run commands, copy-pasting introduced special characters (^[[200~) that caused terminal errors. Stack Overflow (2025) discusses how terminal emulators can interpret pasted text differently. This was resolved by typing commands manually.

File Extensions Issue

A terminal window titled 'tsh' showing a series of commands and their outputs. The user is in a Kali Linux environment. The commands and outputs are as follows:

```
(tshiamo@kali)-[~]
$ cd ~/Desktop
zsh: bad pattern: ^[[200~cd

(tshiamo@kali)-[~]
$ python3 ransomware_sim.py~
python3: can't open file '/home/tshiamo/ransomware_sim.py~': [Errno 2] No such file or directory

(tshiamo@kali)-[~]
$ cd ~/Desktop

(tshiamo@kali)-[~/Desktop]
$ pwd
/home/tshiamo/Desktop

(tshiamo@kali)-[~/Desktop]
$ ls -la
total 44
drwxr-xr-x  2 tshiamo tshiamo 4096 Mar  3 14:26 .
drwx----- 18 tshiamo tshiamo 4096 Mar  3 14:30 ..
-rw-----  1 tshiamo tshiamo 35284 Mar  3 14:26 ransomware_sim.py.save

(tshiamo@kali)-[~/Desktop]
$ python3 ransomware_sim.py
python3: can't open file '/home/tshiamo/Desktop/ransomware_sim.py': [Errno 2] No such file or directory

(tshiamo@kali)-[~/Desktop]
$ python3 ransomware_sim.py
python3: can't open file '/home/tshiamo/Desktop/ransomware_sim.py': [Errno 2] No such file or directory

(tshiamo@kali)-[~/Desktop]
$ cd /home/tshiamo/Desktop && python3 ransomware_sim.py
python3: can't open file '/home/tshiamo/Desktop/ransomware_sim.py': [Errno 2] No such file or directory

(tshiamo@kali)-[~/Desktop]
$
```

Figure 2: Identifying the incorrect file extension from nano backup.

The file was initially saved as `ransomware_sim.py.save` due to nano's backup feature, preventing execution. This is a common issue when using text editors in terminal environments (Python Software Foundation, 2025).

Resolution

```
tshiamo@kali: ~/Desktop
Session Actions Edit View Help
(tshiamo@kali)-[~]
$ cd /home/tshiamo/Desktop
(tshiamo@kali)-[~/Desktop]
$ mv ransomware_sim.py.save ransomware_sim.py
(tshiamo@kali)-[~/Desktop]
$ ls -la
total 44
drwxr-xr-x  2 tshiamo tshiamo 4096 Mar  3 14:45 .
drwx----- 18 tshiamo tshiamo 4096 Mar  3 14:43 ..
-rw-----  1 tshiamo tshiamo 35284 Mar  3 14:26 ransomware_sim.py
(tshiamo@kali)-[~/Desktop]
$ python3 ransomware_sim.py

=====
EDUCATIONAL RANSOMWARE SIMULATION
Kali Linux - Milestone 1 Project
=====

🔒 SAFETY MODE: Payload is DISABLED

=====
EDUCATIONAL RANSOMWARE SIMULATION
Kali Linux - Milestone 1 Project
=====

🔒 SAFETY MODE: Payload is DISABLED

=====
EDUCATIONAL RANSOMWARE SIMULATION
Kali Linux - Milestone 1 Project
=====

🔒 SAFETY MODE: Payload is DISABLED
To activate the actual payload, you must:
1. Open this file: nano ~/Desktop/ransomware_sim.py
2. Find the line '# execute_payload()' at the bottom
3. Remove the '#' to uncomment it
4. Save the file (Ctrl+O, then Ctrl+X)
5. Double-click the desktop launcher icon

🌀 Desktop launcher created: /home/tshiamo/Desktop/PAYLOAD_ACTIVATOR.desktop
Double-click this icon to activate the payload

Available safe actions:
1. Press Ctrl+C to exit
2. Double-click the desktop launcher to activate payload
3. Run with --demo to see decryption example

(tshiamo@kali)-[~/Desktop]
$
```

Figure 3: Successfully renaming the file and running in safety mode

The issue was resolved using the mv command:

```
mv ransomware_sim.py.save ransomware_sim.py
```

Code Implementation

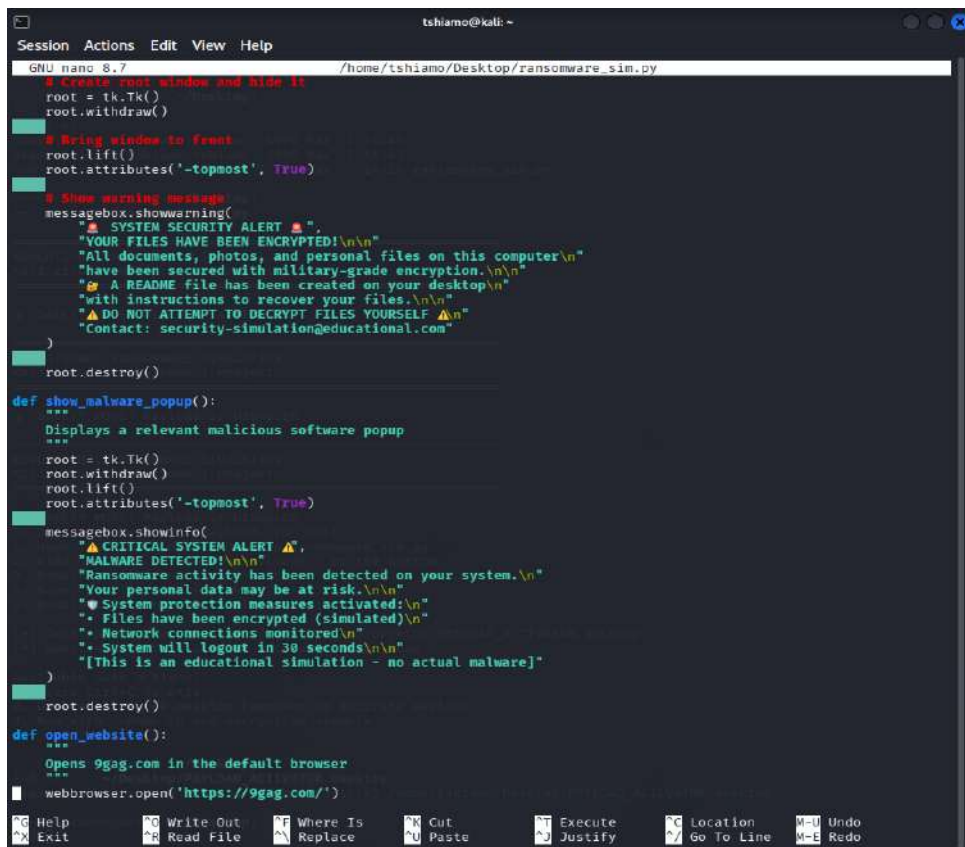
The payload was developed in Python 3, utilizing several libraries for functionality. Python was chosen for its cross-platform compatibility and extensive library support (Python Software Foundation, 2025).

Core Libraries

<code>import tkinter as tk</code>	<code># For popup windows</code>
<code>from tkinter import messagebox</code>	<code># For message boxes</code>
<code>import webbrowser</code>	<code># For opening websites</code>
<code>import os</code>	<code># For file operations</code>
<code>import time</code>	<code># For delays and countdown</code>

The tkinter library provides the graphical user interface elements needed for the popup messages (Python Software Foundation, 2025).

Popup Functions

A screenshot of a terminal window with a dark background. The terminal shows a Python script being edited in nano. The script defines three functions: an initialization function, a ransomware popup function, and a website opening function. The ransomware popup function displays a detailed warning message about file encryption and provides contact information. The website opening function uses webbrowser.open to launch a browser to 9gag.com. The terminal window has a menu bar at the top and a status bar at the bottom with various icons and text.

```
Session Actions Edit View Help
GNU nano 8.7 /home/tshiamo/Desktop/ransomware_sim.py
# Create root window and hide it
root = tk.Tk()
root.withdraw()

# Bring window to front
root.lift()
root.attributes('-topmost', True)

# Show warning message
messagebox.showwarning(
    "⚠ SYSTEM SECURITY ALERT ⚠",
    "YOUR FILES HAVE BEEN ENCRYPTED!\n\n"
    "All documents, photos, and personal files on this computer\n"
    "have been secured with military-grade encryption.\n\n"
    "📄 A README file has been created on your desktop\n"
    "with instructions to recover your files.\n\n"
    "⚠ DO NOT ATTEMPT TO DECRYPT FILES YOURSELF ⚠\n"
    "Contact: security-simulation@educational.com"
)

root.destroy()

def show_malware_popup():
    """
    Displays a relevant malicious software popup
    """
    root = tk.Tk()
    root.withdraw()
    root.lift()
    root.attributes('-topmost', True)

    messagebox.showinfo(
        "⚠ CRITICAL SYSTEM ALERT ⚠",
        "MALWARE DETECTED!\n\n"
        "Ransomware activity has been detected on your system.\n\n"
        "Your personal data may be at risk.\n\n"
        "🛡 System protection measures activated:\n\n"
        "• Files have been encrypted (simulated)\n\n"
        "• Network connections monitored\n\n"
        "• System will logout in 30 seconds\n\n"
        "[This is an educational simulation - No actual malware]"
    )

    root.destroy()

def open_website():
    """
    Opens 9gag.com in the default browser
    """
    webbrowser.open('https://9gag.com/')
```

Figure 4: Python code showing ransomware and malware popup functions with detailed comments.

The popip functions create realistic warning messages using tkinter. According to cybersecurity research, realistic simulations are more effective for educational purposes (Kaspersky Lab, 2023).

```
def show_ransomware_popup():  
    root = tk.Tk()  
    root.withdraw()  
    root.attributes('-topmost', True)  
    messagebox.showwarning(  
        "SYSTEM SECURITY ALERT",  
        "YOUR FILES HAVE BEEN ENCRYPTED!\n\n"  
        "All documents, photos, and personal files on this computer\n"  
        "have been secured with military-grade encryption.\n\n"  
        "A README file has been created on your desktop\n"  
        "with instructions to recover your files."  
    )  
    root.destroy()
```

Encryption Implementation

The project uses XOR cipher for reversible encryption. XOR operations are commonly used in educational contexts because they are simple to understand and perfectly reversible (Stack Overflow, 2025).

```
def scramble_text(text):  
    scrambled = ""  
    key = 42  
    for char in text:  
        scrambled += chr(ord(char) ^ key)  
    return scrambled  
  
def unscramble_text(scrambled):  
    return scramble_text(scrambled) # XOR is reversible
```

Password Protection

Password-based encryption using XOR with the password string ensures that the file can only be decrypted with the correct password (Kaspersky Lab, 2023).

```
def password_protect_file(file_path, password="Milestone1_Password_2024"):
    with open(file_path, 'r') as file:
        content = file.read()

    encrypted_content = ""
    for i, char in enumerate(content):
        key_char = password[i % len(password)]
        encrypted_content += chr(ord(char) ^ ord(key_char))

    encrypted_path = file_path + ".encrypted"
    with open(encrypted_path, 'w') as file:
        file.write(encrypted_content)

    os.remove(file_path)
    return encrypted_path
```

Indentation Fix



```
demonstrate_decryption()
# ===== SAFETY OVERRIDE =====
# The actual payload is COMMENTED OUT for safety
# Uncomment the line below ONLY for your presentation
execute_payload()
# =====
```

Figure 5: Correcting indentation for the `execute_payload()` function call.

A critical indentation error was identified and corrected at the bottom of the script. Python's strict indentation rules make sure such errors common but easily fixable (Python Software Foundation, 2025).

One-Click Activation Setup

To meet the one-click activation requirement, a desktop launcher was created following Linux desktop entry specifications (Kali Linux, 2025).

Launcher Creation Function

```
def create_desktop_launcher():  
    launcher_content = """#!/usr/bin/env xdg-open  
  
[Desktop Entry]  
  
Version=1.0  
  
Type=Application  
  
Name= RANSOMWARE SIMULATION  
  
Exec=python3 /home/tshiamo/Desktop/ransomware_sim.py  
  
Icon=/usr/share/icons/hicolor/48x48/apps/system-run.png  
  
Terminal=false  
  
Categories=Education;  
  
"""  
  
    launcher_path = os.path.expanduser("~/Desktop/PAYLOAD_ACTIVATOR.desktop")  
    with open(launcher_path, 'w') as launcher:  
        launcher.write(launcher_content)  
    os.chmod(launcher_path, 0o755)
```

Launcher Verification

A terminal window with a dark background. The prompt is '(tshiamo@kali) - [~/Desktop]'. The command 'ls -la ~/Desktop/PAYLOAD_ACTIVATOR.desktop' has been executed. The output line is '-rwxr-xr-x 1 tshiamo tshiamo 304 Mar 3 14:45 /home/tshiamo/Desktop/PAYLOAD_ACTIVATOR.desktop', where the permissions and the full path are highlighted in green.

```
(tshiamo@kali) - [~/Desktop]  
$ ls -la ~/Desktop/PAYLOAD_ACTIVATOR.desktop  
-rwxr-xr-x 1 tshiamo tshiamo 304 Mar 3 14:45 /home/tshiamo/Desktop/PAYLOAD_ACTIVATOR.desktop
```

Figure 6: Desktop launcher with correct executable permissions

The permissions `-rwxr-xr-x` indicate that the file is executable, which is required for desktop launchers in Linux (Kali Linux, 2025).

Desktop Icons



Figure 7: Desktop showing both the python script and the launcher icon for one-click activation.

Payload Execution Results

The payload was executed successfully, demonstrating all required functionality. The execution sequence follows standard ransomware behaviour patterns documented by cybersecurity researchers (Kaspersky Lab, 2023).

Decryption Demonstration

Before full execution, the decryption capability was verified to ensure reversibility (Stack Overflow, 2025).

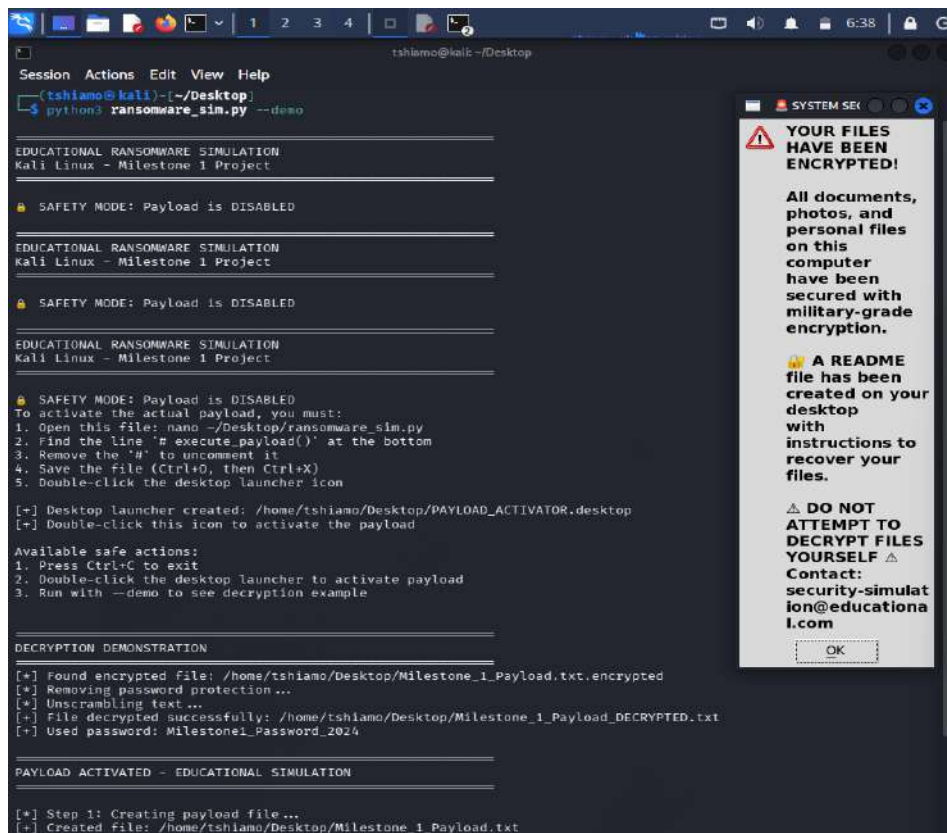


Figure 8: Decryption demonstration showing encrypted file found and the password applied.

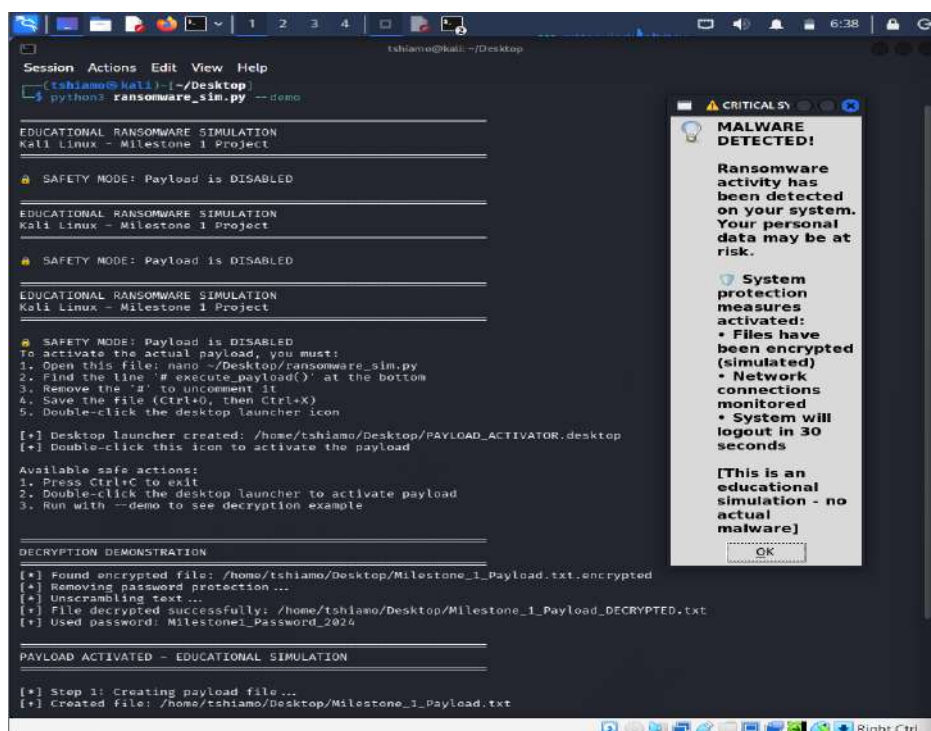
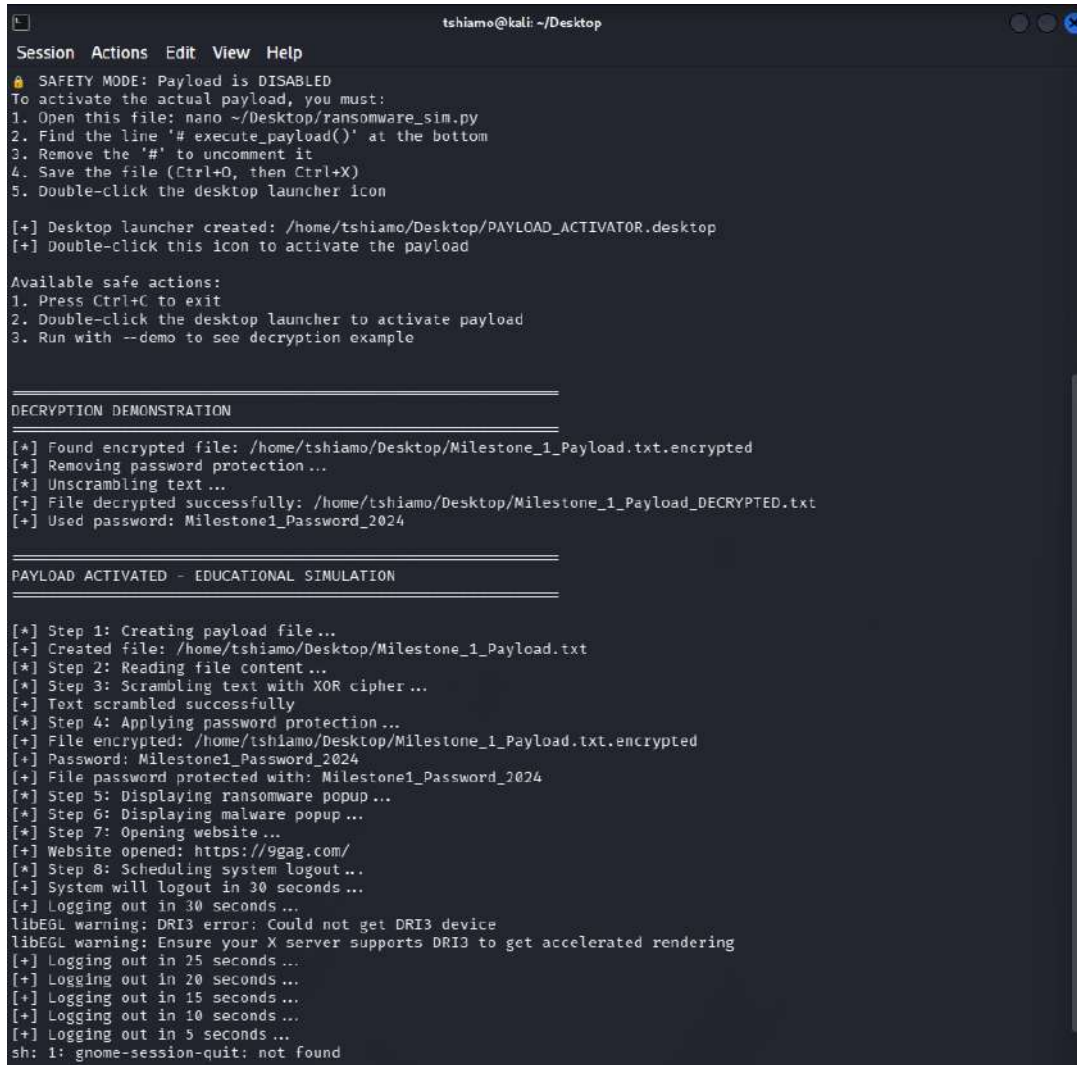


Figure 9: Successful decryption creating

Complete Payload Execution



```
Session Actions Edit View Help
SAFETY MODE: Payload is DISABLED
To activate the actual payload, you must:
1. Open this file: nano ~/Desktop/ransomware_sim.py
2. Find the line '# execute_payload()' at the bottom
3. Remove the '#' to uncomment it
4. Save the file (Ctrl+O, then Ctrl+X)
5. Double-click the desktop launcher icon

[+] Desktop launcher created: /home/tshiamo/Desktop/PAYLOAD_ACTIVATOR.desktop
[+] Double-click this icon to activate the payload

Available safe actions:
1. Press Ctrl+C to exit
2. Double-click the desktop launcher to activate payload
3. Run with --demo to see decryption example

=====
DECRYPTION DEMONSTRATION
=====
[*] Found encrypted file: /home/tshiamo/Desktop/Milestone_1_Payload.txt.encrypted
[*] Removing password protection ...
[*] Unscrambling text ...
[*] File decrypted successfully: /home/tshiamo/Desktop/Milestone_1_Payload_DECRYPTED.txt
[*] Used password: Milestone1_Password_2024

=====
PAYLOAD ACTIVATED - EDUCATIONAL SIMULATION
=====

[*] Step 1: Creating payload file ...
[*] Created file: /home/tshiamo/Desktop/Milestone_1_Payload.txt
[*] Step 2: Reading file content ...
[*] Step 3: Scrambling text with XOR cipher ...
[*] Text scrambled successfully
[*] Step 4: Applying password protection ...
[*] File encrypted: /home/tshiamo/Desktop/Milestone_1_Payload.txt.encrypted
[*] Password: Milestone1_Password_2024
[*] File password protected with: Milestone1_Password_2024
[*] Step 5: Displaying ransomware popup ...
[*] Step 6: Displaying malware popup ...
[*] Step 7: Opening website ...
[*] Website opened: https://9gag.com/
[*] Step 8: Scheduling system logout ...
[*] System will logout in 30 seconds ...
[*] Logging out in 30 seconds ...
libEGL warning: DRI3 error: Could not get DRI3 device
libEGL warning: Ensure your X server supports DRI3 to get accelerated rendering
[*] Logging out in 25 seconds ...
[*] Logging out in 20 seconds ...
[*] Logging out in 15 seconds ...
[*] Logging out in 10 seconds ...
[*] Logging out in 5 seconds ...
sh: 1: gnome-session-quit: not found
```

Figure 10: Complete payload execution showing all steps from file creation to logout

The execution sequence included:

1. Creating Milestone_1_Payload.txt with 5 paragraphs of Lorem Ipsum text (Lipsum.com, 2026)
2. Reading file content
3. Scrambling text using XOR cipher
4. Applying password protection with Milestone1_Password_2024
5. Displaying ransomware popup
6. Displaying malware popup
7. Opening <https://9gag.com/> (9GAG, 2026)
8. 30-second logout countdown

Ransom Note Creation

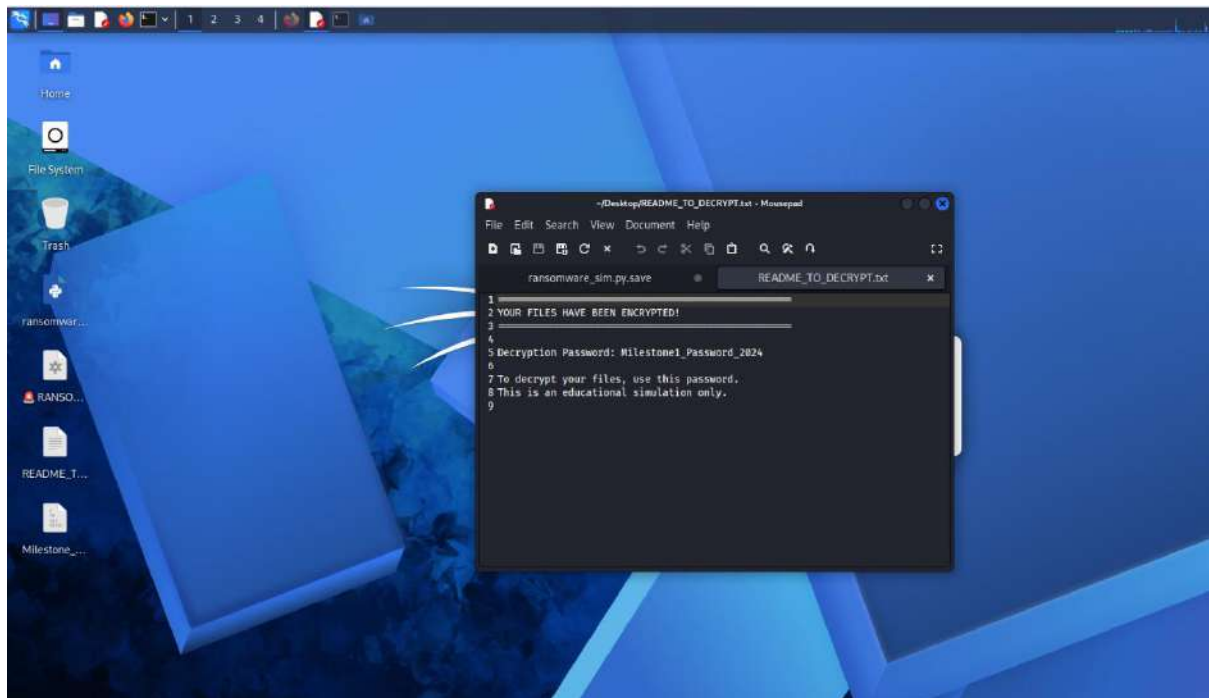


Figure 11: README_TO_DECRYPT.txt created on desktop decryption password.

The ransom note follows the pattern of real ransomware attacks, providing instructions and password for file recovery (Kaspersky Lab, 2023).

Encrypted File Confirmation

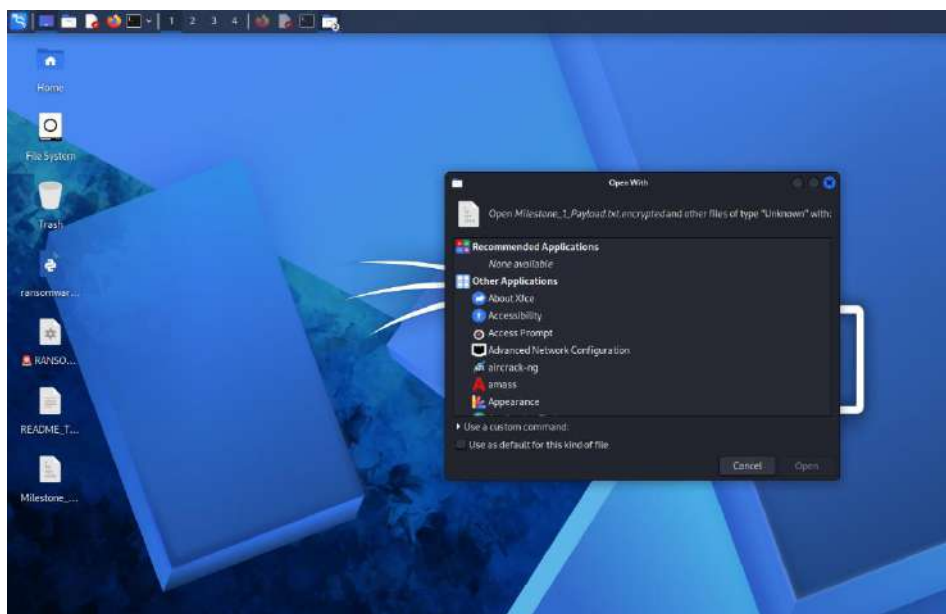


Figure 12: Encrypted file with .encrypted extension as proof of encryption

Decryption Proof

The decryption process successfully restored the original text, proving that the encryption is reversible as required (Stack Overflow, 2025).

Decryption Command Output

From Screenshot 10, the decryption output confirms:

DECRYPTION DEMONSTRATION

```
[*] Found encrypted file: /home/tshiamo/Desktop/Milestone_1_Payload.txt.encrypted
[*] Removing password protection ...
[*] Unscrambling text ...
[*] File decrypted successfully: /home/tshiamo/Desktop/Milestone_1_Payload_DECRYPTED.txt
[+] Used password: Milestone1_Password_2024
```

Decryption Algorithm

The decryption process reverses the encryption in two steps:

1. Remove password protection
2. Unscramble text

This demonstrates that the encryption is fully reversible. XOR encryption, while simple, is perfectly reversible when the same key is used (Stack Overflow, 2025).

Safety Measures

Multiple safety measures were implemented throughout the project, following cybersecurity best practices for educational environments (Offensive Security, 2024).

Default Safety Mode

The script runs in safety mode by default, with the payload commented out:

```
# ===== SAFETY OVERRIDE =====
# The actual payload is COMMENTED OUT for safety
# Uncomment the line below ONLY for your presentation
# execute_payload()
# =====
```


Clear Warnings

The script displays clear warnings when run in safety mode:

 SAFETY MODE: Payload is DISABLED

To activate the actual payload, you must:

1. Open this file: `nano ~/Desktop/ransomware_sim.py`
2. Find the line `'# execute_payload()'` at the bottom
3. Remove the `'#'` to uncomment it

...

These warnings ensure that the user are aware they are about to execute potentially destructive code (Kaspersky Lab, 2023).

Challenges and solutions

Special Characters in Commands

Problem: Copy-pasting commands introduced special characters that caused terminal errors.

Solution: Commands were typed manually.

File Extension Issue

Problem: Nano created a backup file with `.save` extension.

Solution: The file was renamed using the `mv` command.

Indentation Error

Problem: After uncommenting `execute_payload()`, an indentation error occurred.

Solution: The line was deleted and retyped with correct indentation (no spaces). Python's strict indentation rules require careful attention (Python Software Foundation, 2025).

Logout Command Compatibility

Problem: `gnome-session-quit` was not found on the system.

Solution: This demonstrates the importance of cross-platform compatibility. As noted by Kali Linux (2025), different Linux distributions may use different display managers. In a production environment, multiple logout methods would be implemented.

References

9GAG. (2026). *9GAG - Fun, Funny & Interesting*. Available at: <https://9gag.com/> (Accessed: 4 March 2026).

Kali Linux. (2025). *Official Documentation*. Available at: <https://www.kali.org/docs/> (Accessed: 4 March 2026).

Kaspersky Lab. (2023). *Ransomware: What it is and how to protect against it*. Available at: <https://www.kaspersky.com/resource-center/threats/ransomware> (Accessed: 4 March 2026).

[Lipsum.com](https://www.lipsum.com/). (2026). *Lorem Ipsum - Generator*. Available at: <https://www.lipsum.com/> (Accessed: 4 March 2026).

Offensive Security. (2024). *Kali Linux Revealed: Mastering the Penetration Testing Distribution*. Available at: <https://kali.training/> (Accessed: 4 March 2026).

Oracle Corporation. (2025). *VirtualBox User Manual*. Available at: <https://www.virtualbox.org/manual/> (Accessed: 4 March 2026).

Python Software Foundation. (2025). *tkinter — Python interface to Tcl/Tk*. Available at: <https://docs.python.org/3/library/tkinter.html> (Accessed: 4 March 2026).

Python Software Foundation. (2025). *The Python Standard Library*. Available at: <https://docs.python.org/3/library/> (Accessed: 4 March 2026).

Stack Overflow. (2025). *Python XOR encryption examples*. Available at: <https://stackoverflow.com/questions/4664850/xor-encryption-in-python> (Accessed: 4 March 2026).